

Software Architecture and Design Specification

Electronics Store Inventory Management System

Team: 10

Team Members: Vaishnavi Bandaru, Valmiki Uma, Yerramilli Siva Sai Saharsh, Yashas

SRN: PES2UG24CS570, PES2UG24CS573, PES2UG24CS618, PES2UG24CS609

Date: 14-09-2026

Revision History

Version	Date	Author	Change Summary
1.0	14-09-2026	Vaishnavi Bandaru, Valmiki Uma, Yerramilli Siva Sai Saharsh, Yashas	Initial architecture and design specification derived from SRS v1.0

Approvals

Role	Name	Signature / Date
Course Coordinator		
Team Lead		

1. Introduction

1.1 Purpose

This document specifies the architecture and design of the Electronics Store Inventory Management System (ESIMS). Where the SRS states what the system must do, this document states how the software is structured to do it: the components and the interfaces between them, the architectural pattern and why it was chosen, the technology stack and data stores, the runtime behaviour of the principal workflows, the API contracts, and the security design. It is the reference used for implementation, review and test planning.

1.2 Scope

Covers the internal store services defined in SRS Section 1.2: user authentication, product and inventory management, supplier and purchase recording, sales recording with automatic stock updates, low-stock identification, product search, and basic inventory and sales reporting. Excluded, consistent with the SRS: any customer-facing storefront, online payment processing, delivery management, advanced accounting and external e-commerce integration.

1.3 Audience

Project developers, QA and test engineers, store administrators and staff who will operate the system, the course coordinator and evaluators, and future maintainers of the codebase.

1.4 Definitions

Term	Definition
ESIMS	Electronics Store Inventory Management System
SKU	Stock Keeping Unit — the unique identifier of a product
Reorder level	Minimum stock quantity at which replenishment is required
RBAC	Role-Based Access Control — permissions granted by role rather than by individual
ACID	Atomicity, Consistency, Isolation, Durability — the guarantees of a database transaction
ADR	Architecture Decision Record — a short note capturing one decision and its rationale
STRIDE	Threat-modelling taxonomy: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
TLS	Transport Layer Security — encryption for data in transit
DTO	Data Transfer Object — a plain structure exchanged between client and server
Stock movement	An append-only record of every increase or decrease in a product's quantity

2. Document Overview

2.1 How to use this document

Section 3 is the architectural view: read 3.3 to 3.5 first for the component model and the pattern decision, then 3.8 to confirm that every requirement has an owning component. Section 4 is the design view: 4.2 shows how the components collaborate at runtime, and 4.3 gives the contracts a developer implements against. Testers should work from 3.8 (traceability) together with 4.3 and 4.4, since those define the observable behaviour and the error responses that test cases assert on. Security reviewers should read 3.9 with 4.3 and 4.4.

2.2 Related Documents

Document	Relationship to this document
SR v1.0 (06-09-2026)	The source of every requirement referenced here. Requirement IDs (ESIMS-F-###, ESIMS-NF-###, ESIMS-SR-###) are used unchanged.

Document	Relationship to this document
Requirements Traceability Matrix (SRS Section 8)	Extended in Section 3.8 with the owning architectural component for each requirement.
Software Test Plan (STP)	Derives its test suites from the acceptance criteria in SRS Section 6 and the API contracts in Section 4.3 of this document.
Lab 3 component model	Earlier iteration of the component diagram reproduced and refined in Section 3.3.

3. Architecture

3.1 Goals & Constraints

Goals

- **Correct inventory at all times.** Stock must reflect every recorded purchase and sale, and must never go negative (ESIMS-F-008, ESIMS-NF-002).
- **Responsive common operations.** Product search and inventory lookup complete within 3 seconds under normal load (ESIMS-NF-001).
- **Enforced access control.** Only authenticated users act, and administrative functions are restricted by role (ESIMS-SR-001, SR-002).
- **Available during trading hours.** Accessible throughout the store's operating hours except during planned maintenance (ESIMS-NF-003).
- **Modular and maintainable.** Product, inventory, sales, purchase and reporting logic can each be changed with minimal impact on the others (SRS Section 6.5).

Constraints

- Only authorised users may access protected functions; administrative functions are role-restricted.
- Inventory quantities must never become negative; each product must have a unique SKU.
- A relational database must be used, and consistency between product, inventory, purchase and sales records must be maintained.
- Standard desktop or laptop hardware and a modern browser; no specialised hardware (barcode scanners are a future enhancement).
- Delivered by a four-person student team within the course timeline, which rules out operationally heavy designs.

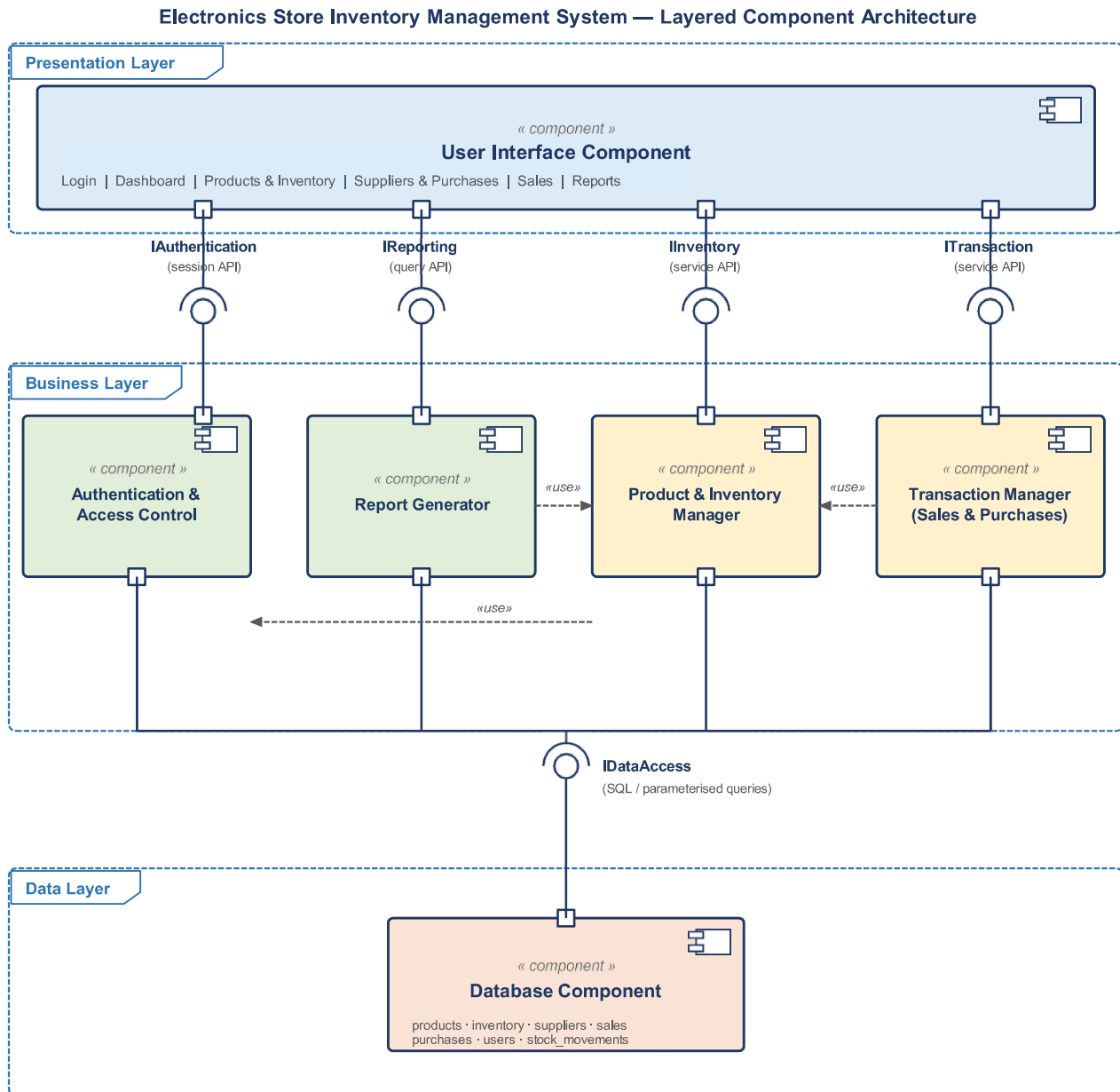
3.2 Stakeholders & Concerns

Stakeholder	Primary concerns	Architectural response
Store Manager / Administrator	Accurate stock, useful reports, control over who can do what	Role-gated write paths; Reporting component with read-only queries; low-stock evaluation on every stock change
Store / Sales Staff	Fast, simple sales entry; clear feedback when something is wrong	Thin presentation tier with a consistent screen set; one uniform error format returned by every endpoint
Store owner / business	No lost or oversold stock; trustworthy sales figures	Single-database ACID transactions; append-only stock movement audit log; totals computed server-side
Developers (Team 10)	Modularity, parallel work, testability	One service per functional area behind a narrow interface; strict layering
Testers / evaluators	Every requirement verifiable and traceable	Traceability matrix in Section 3.8; API contracts and error codes in Sections 4.3 and 4.4
Future maintainers	Ability to change one area safely	Cross-cutting concerns centralised; database reachable only through the data-access layer

Table 1. Stakeholders, their concerns and how the architecture addresses each.

3.3 Component (UML) Diagram

The system is decomposed into six components arranged in three layers. Provided interfaces are shown in ball notation, required interfaces in socket notation, and internal usage dependencies as dashed «use» arrows. Every call crosses exactly one layer boundary, and the database is reachable only through `IDataAccess`.



Assembly connectors (ball-in-socket) show provided and required interfaces. Dashed «use» arrows show internal usage dependencies. No business component touches the database directly — all access is through `IDataAccess`.

Figure 1. UML component diagram for ESIMS — six components, five interfaces, three usage dependencies.

3.4 Component Descriptions

Component	Layer	Responsibility	Provided / required interfaces
User Interface Component	Presentation	Renders the six screens defined in SRS Section 3.1, collects input and displays stock, alerts and reports. Contains no business rules and holds no database credentials.	Requires: <code>IAuthentication</code> , <code>IInventory</code> , <code>ITransaction</code> , <code>IReporting</code>
Authentication & Access Control Component	Business	Verifies credentials against salted password hashes, creates and invalidates sessions, resolves the caller's role and authorises every protected operation.	Provides: <code>IAuthentication</code> Requires: <code>IDataAccess</code>

Component	Layer	Responsibility	Provided / required interfaces
Product & Inventory Manager Component	Business	Owns products, categories and stock. The only component permitted to change a quantity; rejects any change that would make stock negative; evaluates reorder levels; performs product search by name or SKU.	Provides: IInventory Requires: IDataAccess, IAuthentication
Transaction Manager Component	Business	Records sales and purchases with their line items, maintains supplier records, computes sale totals from stored unit prices, and drives the matching stock change inside a single transaction.	Provides: ITransaction Requires: IInventory, IDataAccess
Report Generator Component	Business	Produces the inventory report and the date-ranged sales report as read-only queries. Never writes.	Provides: IReporting Requires: IInventory, IDataAccess
Database Component	Data	Persists all data and enforces integrity at the schema level (unique SKU, non-negative stock, foreign keys); provides transactional read and write access.	Provides: IDataAccess

Table 2. Responsibilities and interfaces of each component.

3.5 Chosen Architecture Pattern and Rationale

Layered (three-tier) architecture was chosen, deployed in a client-server topology: browsers on staff workstations talk to one application server, which is the only client of the database server.

Why layered

- **Inventory consistency is the hardest requirement and layering keeps it local.** All business rules sit in one tier over a single database, so a sale and its stock decrease run inside one local ACID transaction and stock is mutable only through the Inventory component. This makes ESIMS-F-008 and ESIMS-NF-002 enforceable at a single code path rather than at every caller.
- **The scenario maps directly onto three tiers.** The six screens are the presentation tier, the functional areas of SRS Section 4 are the business tier, and the required relational database is the data tier.
- **Security gets one choke point.** Authentication and role checks are applied at the boundary between the presentation and business tiers, so they cannot be bypassed by calling an endpoint directly.
- **It fits the team and the timeline.** Narrow interfaces let four developers work in parallel with familiar tooling and no distributed-systems operational burden.

Alternatives considered

Pattern	Assessment for ESIMS	Decision
Microservices	Would split sales and inventory into separately deployed services with separate stores, turning the core consistency requirement into a distributed-transaction or saga problem. A single store with a handful of concurrent users has no scaling pressure that would repay the operational complexity and network latency.	Rejected
Client-Server (alone)	Correctly describes where code runs, and ESIMS is genuinely client-server at the deployment level. But it says nothing about how server-side code is organised, so on its own it gives no separation between UI, business rules and data access.	Adopted as the deployment model within the layered design
Event-driven	Asynchronous stock updates would make the moment at which stock is correct non-deterministic, which conflicts directly with ESIMS-F-008.	Rejected

Table 3. Alternative patterns considered and the reason each was set aside.

Layering rules

- A layer may call only the layer immediately beneath it; the UI never issues SQL and controllers never write to tables directly.
- Cross-cutting concerns — access control, input validation, logging, error handling — are applied at layer boundaries rather than scattered through business code.
- Transactions begin and end only in the data-access layer.

3.6 Technology Stack & Data Stores

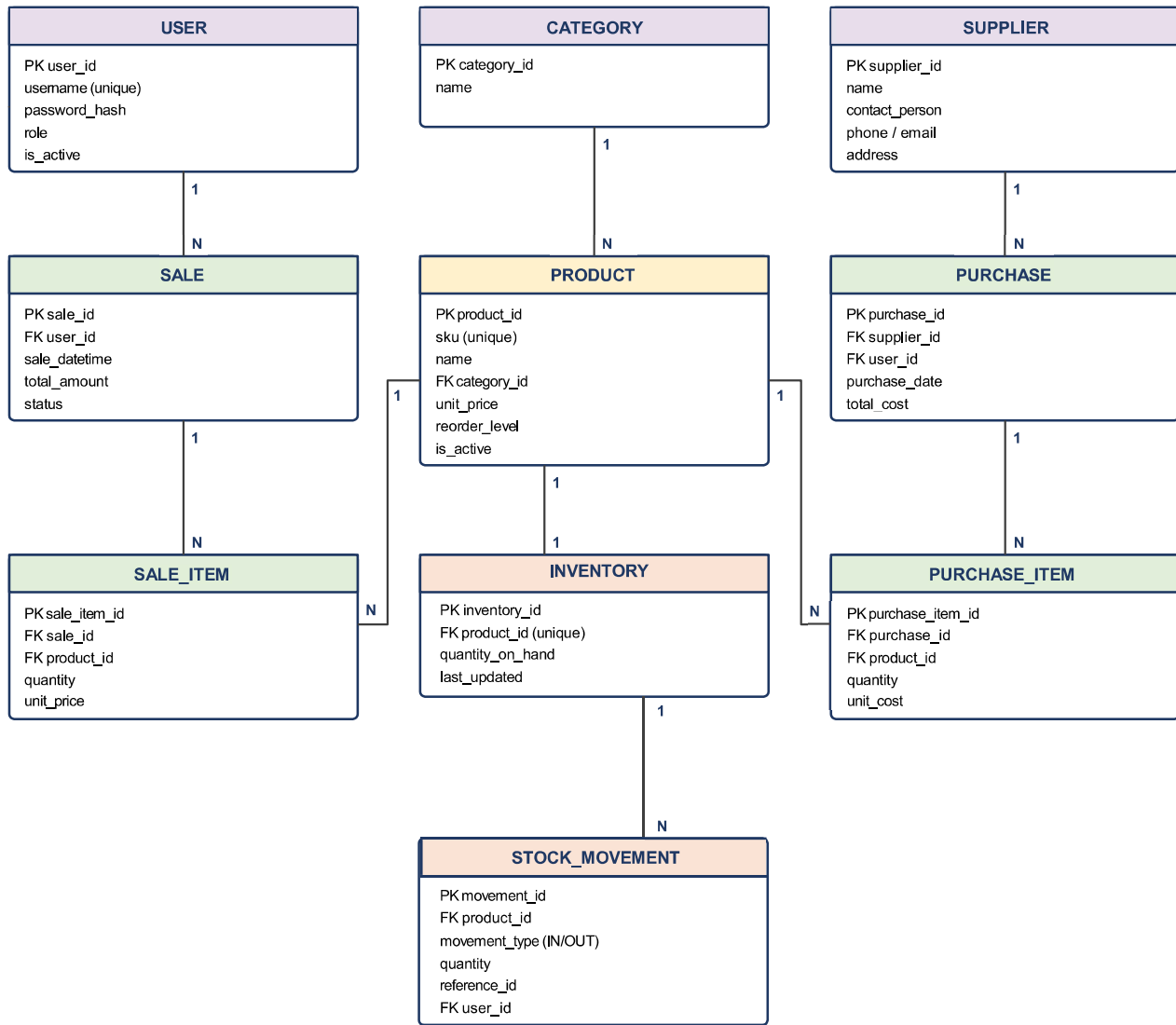
SRS Section 2.4 defers the concrete technology to the design phase. The stack below is the recommended instantiation; the right-hand column states what the architecture actually requires, so any substitution can be evaluated against it.

Layer	Recommended technology	What the architecture requires
Presentation	HTML5, CSS, JavaScript (optionally React) in a modern browser	A thin client that renders data and collects input and holds no business rules
API	REST over HTTPS with JSON payloads (Flask, Express or Spring Boot)	A stateless request handler that authenticates, authorises and validates before delegating
Business logic	Service classes in the same language as the API framework	One module per functional area with an explicit public interface
Data access	ORM or parameterised SQL with explicit transaction boundaries	Parameterised statements only; programmatic control over commit and rollback
Database	MySQL 8 or PostgreSQL 15	ACID transactions, foreign keys, UNIQUE and CHECK constraints, indexing
Security	Server-side sessions or signed tokens; bcrypt or Argon2 hashing;	Credentials never stored, logged or transmitted in plaintext
Deployment	Application server and database server on the store LAN	The database must not be directly reachable from client machines

Table 4. Technology stack and the constraint each choice must honour.

Data stores

A single relational database holds all persistent state, normalised into ten tables. Products and inventory are kept separate because the catalogue is read often and written rarely while inventory is written on every transaction. Purchases and sales each use a header-plus-line-items pair, which is what allows one sale to contain several products (ESIMS-F-012).



Legend: PK = primary key, FK = foreign key. 1 : N denotes a one-to-many relationship.
 INVENTORY holds exactly one row per PRODUCT. STOCK_MOVEMENT is an append-only audit log of every stock change.

Figure 2. Logical data model. INVENTORY holds one row per product; STOCK_MOVEMENT is an append-only audit of every quantity change.

Table	Purpose	Key integrity rules
users	Store personnel and their roles	username UNIQUE; role in (Administrator, Store Staff); password_hash never plaintext
categories	Product grouping	name UNIQUE
products	Catalogue of electronic products	sku UNIQUE; unit_price ≥ 0 ; reorder_level ≥ 0
inventory	Current quantity per product	product_id UNIQUE FK; quantity_on_hand ≥ 0 enforced by CHECK
suppliers	Supplier contact details	name required; referenced by purchases
purchases / purchase_items	Purchase header and line items	quantity > 0; unit_cost ≥ 0 ; line items cascade with the header
sales / sale_items	Sale header and line items	quantity > 0; unit_price captured at time of sale; total_amount derived server-side, never client-supplied
stock_movements	Audit log of every stock change	Append-only; movement_type in (IN, OUT); records user_id and timestamp

Table 5. Data stores and the integrity rules enforced at schema level.

Indexing and derivation decisions

- **Current quantity is stored, not derived.** Recomputing stock from all movements on each read would grow slower over time and endanger ESIMS-NF-001; stock_movements exists for audit and reconciliation, not routine reads.
- **Indexes.** products.sku (unique), products.name, inventory.product_id, sales.sale_datetime and purchases.purchase_date, supporting search (ESIMS-F-006) and date-ranged reporting (ESIMS-F-016).
- **Soft deletion.** Products and users are deactivated via an is_active flag rather than deleted, so historical transactions keep valid foreign keys.

3.7 Risks & Mitigations

Risk	Impact	Mitigation
Two staff sell the last unit of a product at the same time	Oversold stock; inventory goes negative, violating ESIMS-F-008	The inventory row is locked for update inside the transaction, so the second transaction sees the updated quantity and is rejected. A database CHECK constraint acts as a backstop.
Database node fails	Store operations halt until recovery	Scheduled backups with a documented restore procedure. Acceptable because ESIMS-NF-003 scopes availability to store operating hours.
Business logic leaks into the UI or controllers	Rules become untestable and inconsistent; maintainability degrades	Layering rules enforced at code review; services expose the only write paths; the UI holds no database credentials.
Report queries scan large transaction tables as data grows	Response time exceeds the 3-second budget	Indexes on transaction dates, mandatory date-range bounds and server-side pagination on report endpoints.
Role checks implemented only in the UI	Privilege escalation by calling the API directly	Authorisation enforced server-side on every endpoint; role read from the session, never from the client payload.
Partial write leaves a sale recorded without the stock decrease	Inventory and sales records disagree	All multi-table writes run in one transaction; any failure rolls the whole operation back (ESIMS-NF-002).
Scope creep toward barcode scanning or e-commerce	Timeline risk for a four-person team	Both are recorded as out of scope in Section 4.6 and attach at defined extension points rather than requiring structural change.

Table 6. Architectural risks, their impact and mitigations.

3.8 Traceability to Requirements

This matrix extends the RTM in SRS Section 8 with the architectural element responsible for each requirement.

Req ID	Requirement	Responsible component	Test case
ESIMS-F-001	User login	Authentication & Access Control → IDataAccess	TC-Auth-01
ESIMS-F-002	Invalid login handling	Authentication & Access Control (hash comparison, generic error)	TC-Auth-02
ESIMS-F-003	Add product	Product & Inventory Manager	TC-Prod-01
ESIMS-F-004	Update product	Product & Inventory Manager	TC-Prod-02
ESIMS-F-005	View stock	Product & Inventory Manager (IInventory.getStock)	TC-Inv-01
ESIMS-F-006	Search products	Product & Inventory Manager (indexed name / sku query)	TC-Inv-02
ESIMS-F-007	Low-stock detection	Product & Inventory Manager (quantity ≤ reorder_level)	TC-Inv-03
ESIMS-F-008	Prevent negative stock	Product & Inventory Manager + CHECK constraint on inventory	TC-Inv-04
ESIMS-F-009	Manage suppliers	Transaction Manager (supplier records)	TC-Sup-01
ESIMS-F-010	Record purchase	Transaction Manager → IDataAccess (transactional)	TC-Pur-01
ESIMS-F-011	Update stock after purchase	Transaction Manager → IInventory.increaseStock	TC-Pur-02
ESIMS-F-012	Record sale	Transaction Manager (header + line items)	TC-Sale-01
ESIMS-F-013	Calculate sale total	Transaction Manager (server-side from stored prices)	TC-Sale-02
ESIMS-F-014	Update stock after sale	Transaction Manager → IInventory.decreaseStock	TC-Sale-03
ESIMS-F-015	Inventory report	Report Generator (read-only query)	TC-Report-01
ESIMS-F-016	Sales report	Report Generator (date-ranged, indexed)	TC-Report-02
ESIMS-NF-001	Response time	Indexing, stored current quantity	TC-Perf-01
ESIMS-NF-002	Inventory consistency	Database Component transaction management; row locking	TC-Rel-01
ESIMS-NF-003	Availability	Stateless API; independent restart; scheduled maintenance	TC-Avail-01
ESIMS-NF-004	Usability	User Interface Component; unified error response format	TC-UX-01
ESIMS-NF-005	Data protection	Access control + password hashing	TC-Sec-01
ESIMS-SR-001	Authentication	Access-control boundary on every protected endpoint	TC-Sec-02
ESIMS-SR-002	Access restriction	Server-side role check per endpoint	TC-Sec-03
ESIMS-SR-003	Password hashing	Authentication Component (salted bcrypt / Argon2)	TC-Sec-04
ESIMS-SR-004	Input validation	Validation at the API boundary + parameterised statements	TC-Sec-05
ESIMS-SR-005	Transaction protection	Role-gated writes; append-only stock_movements; soft deletion	TC-Sec-06

Table 7. Requirement-to-component traceability.

3.9 Security Architecture

Security objectives

The two objectives stated in SRS Section 5.1.1 drive the design. **Confidentiality**: user credentials and store transaction information are accessible only to authorised personnel. **Integrity**: product, inventory, purchase and sales information cannot be modified by anyone not authorised to do so.

Threat modelling (STRIDE)

Threat	Scenario in ESIMS	Mitigation	Req ID
Spoofing	One staff member logs in as another, or an outsider guesses a password	Individual accounts; salted bcrypt/Argon2 hashes; server-side sessions; progressive delay after repeated failed logins; generic failure message that does not reveal whether the username exists	SR-001, SR-003
Tampering	Direct modification of a stock quantity or a recorded sale	Inventory is mutable only through the Inventory component; all write paths are role-gated; CHECK and FOREIGN KEY constraints at schema level; totals recomputed server-side rather than accepted from the client	SR-005, F-008, F-013
Repudiation	A user denies having recorded a sale or adjusted stock	Every stock movement stores user_id and timestamp in an append-only table; application logs record who invoked each write operation	SR-005, NF-002
Information disclosure	Credentials or transaction data exposed in transit, in logs or in errors	Password column never returned by any API or written to logs; internal exceptions logged in full but returned to the client as a generic message so schema details are not leaked	NF-005, SR-003
Denial of service	Repeated login attempts or unbounded report queries exhaust the server	Mandatory date bounds on report endpoints.	NF-001, NF-003
Elevation of privilege	A Store Staff user calls an administrator-only endpoint directly	Role is read from the server-side session, never from the request; each endpoint declares its required role; UI hiding of unavailable actions is treated as convenience, not a control	SR-002

Table 8. STRIDE threat model and the control that mitigates each threat.

Trust boundaries

Two trust boundaries exist. Between the browser and the application server, nothing arriving from the client is trusted — including role claims, prices and totals — which is why a sale total is always recomputed server-side from stored unit prices. Between the application server and the database, the application holds the only database credentials, and those credentials grant access to the ESIMS schema only.

4. Design

4.1 Design Overview

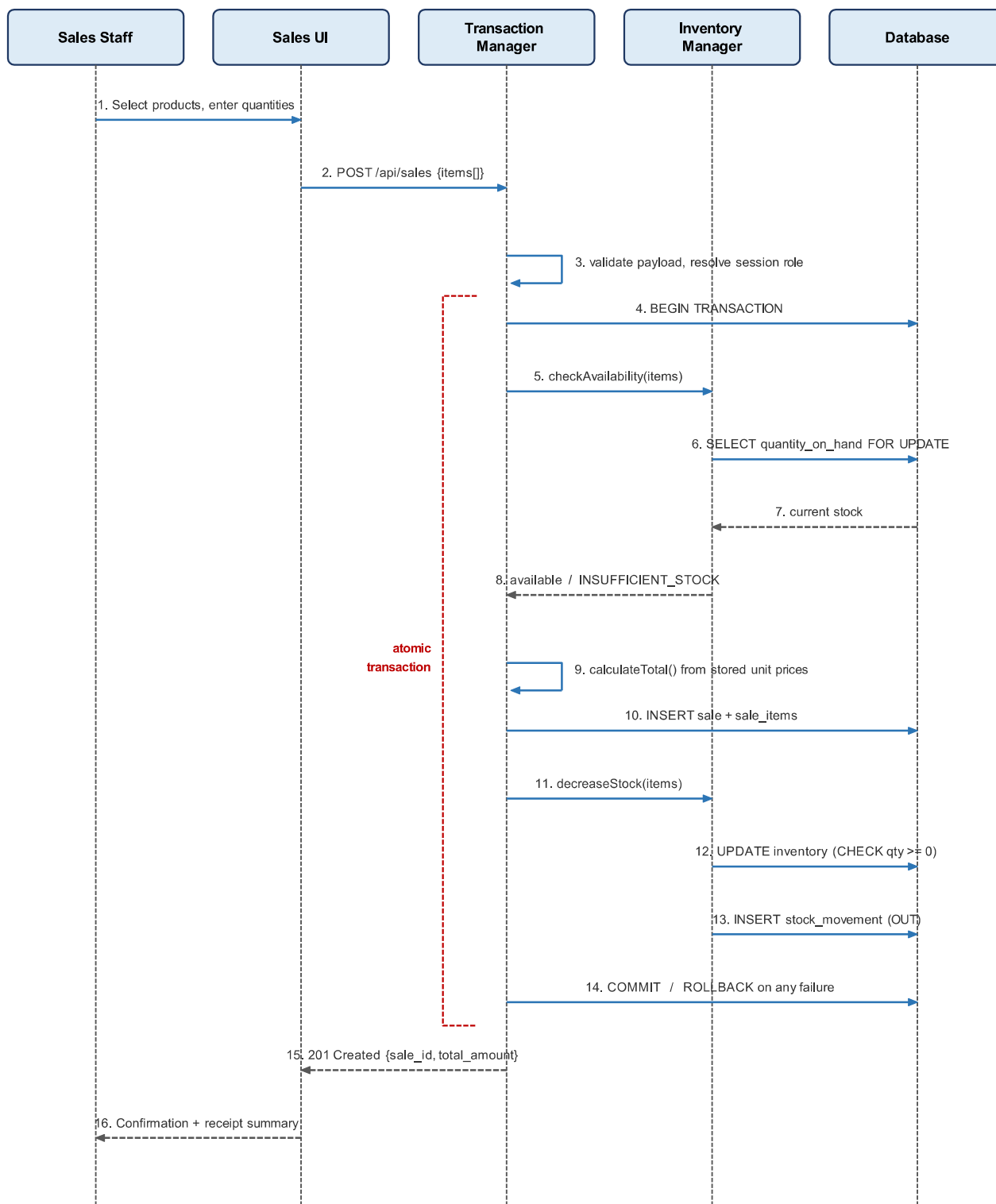
ESIMS is designed as layered modules so that each concern has exactly one home. The presentation tier renders and collects; the business tier holds every rule; the data tier persists and enforces integrity. Within the business tier, each component owns one functional area and exposes a narrow interface, so a change to reporting does not touch sales, and a change to how stock is decremented is made in one place.

The design is deliberately shaped around one invariant: **no component other than the Product & Inventory Manager may change a quantity**. Every workflow that affects stock — recording a sale, recording a purchase, correcting an error — routes through that component, which checks authorisation, applies the non-negative rule and writes an audit record. The sequence diagrams below show this for the two principal flows.

4.2 UML Sequence Diagrams

4.2.1 Flow 1 — Recording a sale (ESIMS-F-012, F-013, F-014)

This is the most correctness-critical path in the system. Stock is checked and decremented inside the same transaction that writes the sale, so a sale can never be recorded without the matching stock decrease and stock can never fall below zero.

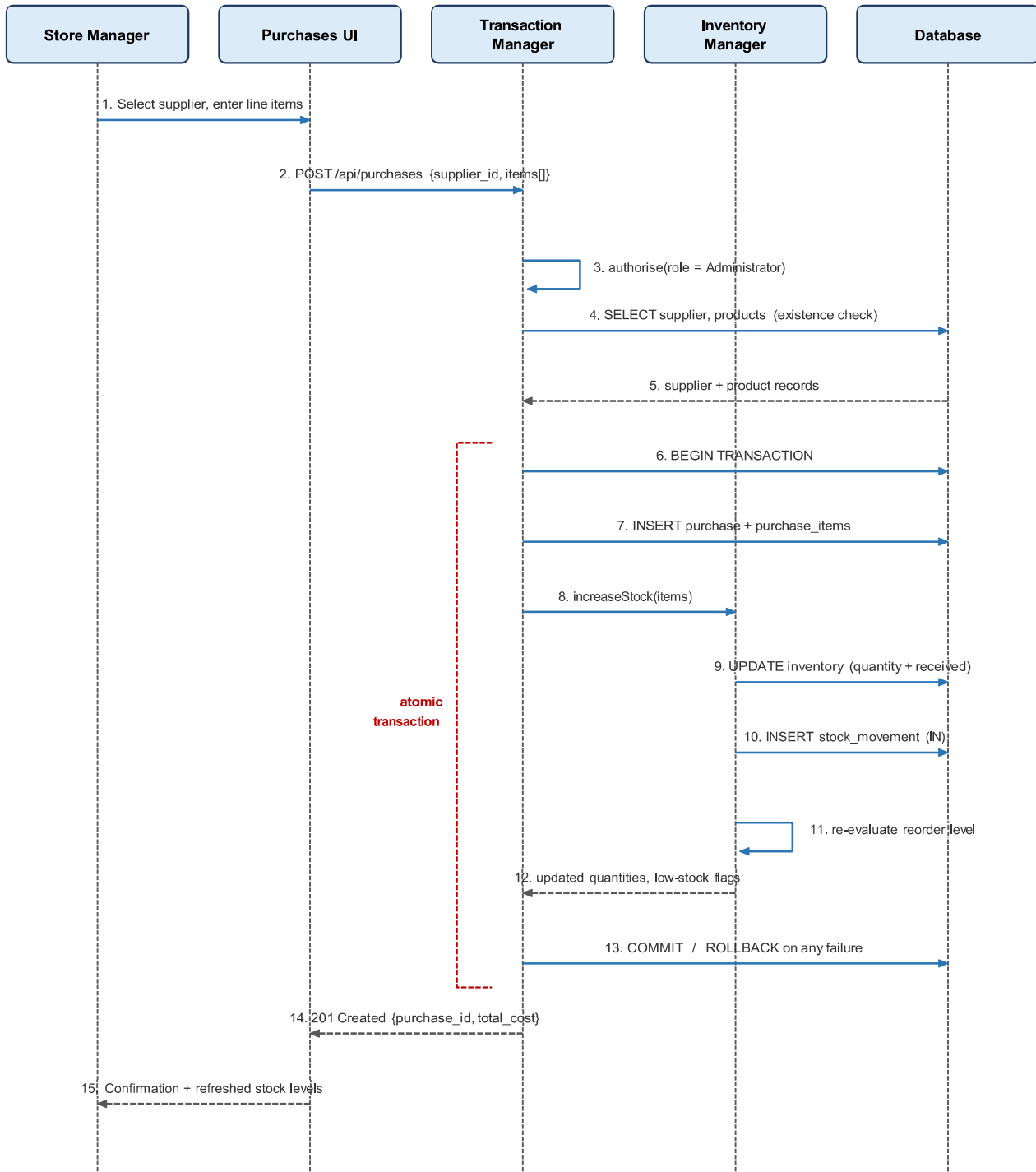


Steps 4-14 execute as one database transaction, so a failed stock check or write leaves inventory unchanged (ESIMS-F-008, ESIMS-NF-002). Solid arrows = synchronous calls, dashed = returns, looped arrows = self-calls.

Figure 3. Sequence diagram — recording a sale, showing the transaction boundary.

4.2.2 Flow 2 — Recording a purchase from a supplier (ESIMS-F-010, F-011)

The purchase flow mirrors the sale flow but increases stock. It is restricted to the Administrator role, and the reorder-level evaluation runs inside the same transaction so the dashboard's low-stock alert clears as soon as the purchase commits.



Purchase recording (ESIMS-F-010, F-011) mirrors the sales flow but increases stock. Low-stock flags are recalculated inside the same transaction so the dashboard alert clears as soon as the purchase commits.

Figure 4. Sequence diagram — recording a purchase from a supplier.

4.3 API Design

Interfaces are defined below for three components: the Transaction Manager, the Product & Inventory Manager and the Authentication & Access Control component. All endpoints exchange JSON over HTTPS. Every endpoint except login requires an authenticated session; the role column in the summary table states who may call each one.

4.3.1 Transaction Manager Component

Endpoint	/api/sales
Method	POST
Role	Store Staff or Administrator
Request	{ "items": [{ "product_id": 12, "quantity": 2 }, { "product_id": 47, "quantity": 1 }] }
Response	201 { "sale_id": 3391, "total_amount": 27450.00, "lines": [...], "sale_datetime": "..." }
Errors	400 VALIDATION_FAILED · 401 AUTH_REQUIRED · 404 PRODUCT_NOT_FOUND · 409 INSUFFICIENT_STOCK (names the product and available quantity) · 500 INTERNAL_ERROR
Notes	total_amount is computed server-side from stored unit prices and is ignored if supplied by the client (ESIMS-F-013). The whole call is one transaction.

Endpoint	/api/purchases
Method	POST
Role	Administrator only
Request	{ "supplier_id": 5, "items": [{ "product_id": 12, "quantity": 20, "unit_cost": 9800.00 }] }
Response	201 { "purchase_id": 812, "total_cost": 196000.00, "updated_stock": [...] }
Errors	400 VALIDATION_FAILED · 401 AUTH_REQUIRED · 403 FORBIDDEN · 404 SUPPLIER_NOT_FOUND · 500 INTERNAL_ERROR

4.3.2 Product & Inventory Manager Component

Endpoint	/api/products?q={term}
Method	GET
Role	Store Staff or Administrator
Request	q matches product name or SKU; page is 1-based; page size is fixed server-side
Response	200 { "results": [{ "product_id": 12, "sku": "LG-TV-55", "name": "...", "unit_price": 54900.00, "quantity_on_hand": 7 }] }
Errors	400 VALIDATION_FAILED · 401 AUTH_REQUIRED

Endpoint	/api/inventory/low-stock
Method	GET
Role	Store Staff or Administrator
Response	200 { "count": 3, "results": [{ "product_id": 47, "name": "...", "quantity_on_hand": 2, "reorder_level": 5 }] }
Errors	401 AUTH_REQUIRED
Notes	Returns products where quantity_on_hand < reorder_level (ESIMS-F-007). Consumed by the Dashboard on load and after every stock-changing operation.

4.3.3 Authentication & Access Control Component

Endpoint	/api/auth/login
Method	POST
Role	Public
Request	{ "username": "rmenon", "password": "... " }
Response	200 { "user_id": 4, "username": "rmenon", "role": "Administrator" } + session cookie
Errors	401 AUTH_INVALID (identical message whether the username or the password is wrong) · 429 TOO_MANY_ATTEMPTS
Notes	The password is compared against a salted hash and is never stored, returned or logged (ESIMS-SR-003).

4.3.4 Endpoint summary

Method & path	Component	Role	Req ID
POST /api/auth/login	Authentication	Public	F-001, F-002
POST /api/auth/logout	Authentication	Any	SR-001
GET /api/products	Product & Inventory	Any	F-006
POST /api/products	Product & Inventory	Admin	F-003
PUT /api/products/{id}	Product & Inventory	Admin	F-004
GET /api/inventory	Product & Inventory	Any	F-005
GET /api/inventory/low-stock	Product & Inventory	Any	F-007
GET / POST / PUT /api/suppliers	Transaction Manager	Admin	F-009
POST /api/purchases	Transaction Manager	Admin	F-010, F-011
POST /api/sales	Transaction Manager	Any	F-012, F-013, F-014
GET /api/reports/inventory	Report Generator	Any	F-015
GET /api/reports/sales?from&to	Report Generator	Admin	F-016

4.4 Error Handling, Logging & Monitoring

Standardised error responses

Every failure returns the same JSON shape — a machine-readable code, a human-readable message and, where applicable, the offending field — so the UI can render feedback without parsing prose and test cases can assert on the code rather than the wording.

HTTP	Code	When it is returned	What the user sees
400	VALIDATION_FAILED	Type, range, required-field or format check fails at the API boundary	Inline message against the specific field
401	AUTH_REQUIRED	No valid session on a protected endpoint	Redirect to the login screen
401	AUTH_INVALID	Wrong username or password	Generic "invalid credentials" message
403	FORBIDDEN	Authenticated but the role does not permit the operation	Explanatory message, no data
404	PRODUCT_NOT_FOUND / SUPPLIER_NOT_FOUND	Referenced record does not exist or is inactive	Named record message
409	DUPLICATE_SKU	A product with the same SKU already exists	Field-level message on the SKU field
409	INSUFFICIENT_STOCK	A sale would drive a quantity below zero	Names the product and the quantity available
429	TOO_MANY_ATTEMPTS	Repeated failed logins from the same source	Retry-after message
500	INTERNAL_ERROR	Unhandled exception	Generic message; details go to the log only

Logging

- No sensitive information is logged: passwords, password hashes and session tokens never appear in any log line.
- Every write operation logs the user, the operation, the affected record and a timestamp, which together with the append-only stock_movements table provides the audit trail required by ESIMS-SR-005.
- Failed authentication attempts are logged with the username and source, but never with the submitted password.
- Internal exceptions are logged with a full stack trace and correlated to the generic client response by a request identifier, so support can trace a reported failure without the error message disclosing schema detail.

Monitoring

Monitoring is out of scope for the student implementation; see Section 4.6

4.5 UX Design

The interface is built for staff who are not technical users and who work under time pressure at the counter. The design principles below follow from ESIMS-NF-004 and from SRS Section 6.3.

- **Six screens, one grammar.** Login, Dashboard, Products & Inventory, Suppliers & Purchases, Sales and Reports share the same layout, table behaviour, button placement and confirmation pattern, so a skill learned on one screen transfers to the others.
- **Dashboard-first.** Low-stock items, recent sales and recent purchases are surfaced on load, so the manager does not have to run a query to learn what needs attention.
- **Keyboard-first sales entry.** The Sales screen supports typing a SKU and quantity and moving between fields without the mouse, because sales entry is the highest-frequency task in the store.
- **Confirmation before stock-affecting actions.** Recording a sale or a purchase shows a summary with the computed total before it is committed, since these actions change inventory and are not casually reversible.
- **Feedback in one consistent place.** Every error from Section 4.4 renders in the same position and style, with field-level messages attached to the field that caused them.
- **Low stock is not signalled by colour alone.** Low-stock rows carry a text label and an icon as well as colour, so the signal survives colour-vision deficiency and monochrome printing.
- **Role-appropriate screens.** Store Staff do not see administrative controls. This is a usability measure only — the authoritative check is always server-side (Section 3.9).
- **Accessible defaults.** Readable base font size, sufficient contrast, visible focus indicators and labelled form controls throughout.

4.6 Open Issues & Next Steps

Item	Status	Notes
Choice of framework and database	Open	Section 3.6 recommends a stack; the final selection is to be confirmed with the course coordinator before implementation begins.
Receipt generation	Open	The use-case diagram in SRS Section 7 includes "Generate receipt", but no numbered requirement covers it. Either add a requirement to the SRS or record it as out of scope for v1.0.
Barcode scanning	Future enhancement	Attaches at the presentation layer as a keyboard-emulating input device for the Sales and Products screens; no architectural change required.
Multi-location inventory	Future enhancement	Would add a location dimension to the inventory table, affecting only the Product & Inventory Manager and its data access.
Automated purchase-order generation	Future enhancement	A new service consuming the existing low-stock query; no change to existing components.
Tax and discount handling	Out of scope for v1.0	sale_items already captures unit_price at the time of sale, which is the extension point if this is added.
Accounting or e-commerce integration	Out of scope	Would be added as a further client of the existing REST API rather than as a new path into the database.
TLS/HTTPS	Deferred to deployment	Not enforced in the student build; see 3.6, 3.9, 3.8 (NF-005).
Rate limiting	Deferred to deployment	Not implemented; see 3.9 (DoS row).
Monitoring/alerting	Future enhancement	Metrics table moved here from 4.4.
agination on list/report endpoints	Deferred	Dataset size doesn't require it for this deployment.
Connection pooling	Deferred	Not needed at this scale

Metrics table moved here from 4.4.

Metric	Why it is monitored	Alert condition
Transaction rollback rate	A rising rate indicates a defect in the sale or purchase path or a data problem	More than 2% of write operations roll back over an hour
p95 response time for search and inventory lookup	Directly verifies ESIMS-NF-001	p95 exceeds 3 seconds
Failed login rate	Detects credential-guessing attempts (STRIDE: Spoofing)	More than 10 failures from one source in 5 minutes
Application uptime during store hours	Verifies ESIMS-NF-003	Any unplanned outage during operating hours
Database connection errors	Early warning of data-tier failure	Any sustained connection failure
Low-stock item count	Operational signal for the manager rather than a fault signal	Reported on the dashboard, not alerted

5. Appendices

5.1 Glossary

Term	Definition
ACID	Atomicity, Consistency, Isolation, Durability — the guarantees provided by a database transaction
ADR	Architecture Decision Record — a short note capturing one decision, its context and its consequences
Assembly connector	UML notation joining a provided interface (ball) to a required interface (socket)
CRUD	Create, Read, Update, Delete
ESIMS	Electronics Store Inventory Management System
RBAC	Role-Based Access Control
Reorder level	The stock quantity at or below which a product is flagged for replenishment
SKU	Stock Keeping Unit — the unique identifier of a product
STRIDE	Threat taxonomy: Spoofing, Tampering, Repudiation, Information disclosure, Denial of service, Elevation of privilege
Stock movement	An append-only record of a single increase or decrease in a product's quantity
TLS	Transport Layer Security

5.2 References

Reference	Use in this document
Electronics Store Inventory Management System — SRS v1.0, Team 10, 06-09-2026	Source of all requirements, constraints and acceptance criteria referenced here
ISO/IEC/IEEE 42010:2011 — Systems and software engineering: Architecture description	Structure of the architectural description: stakeholders, concerns and views
ISO/IEC/IEEE 29148 — Requirements engineering	Requirement identification and traceability conventions
OWASP Top Ten and OWASP Application Security Verification Standard	Input validation, authentication, session management and injection-prevention controls in Section 3.9
OWASP Password Storage Cheat Sheet	Choice of salted bcrypt / Argon2 hashing for ESIMS-SR-003
Microsoft STRIDE threat-modelling methodology	Structure of the threat model in Section 3.9
OMG Unified Modeling Language Specification, v2.5.1	Notation for the component and sequence diagrams in Sections 3.3 and 4.2
NIST SP 800-160 — Systems Security Engineering	Security-by-design framing of trust boundaries

5.3 Tools

Tool	Purpose
draw.io / StarUML	Authoring the UML component and sequence diagrams
PlantUML	Text-based regeneration of sequence diagrams during development
Git and GitHub (SELABS repository)	Version control and submission
MySQL Workbench / pgAdmin	Schema design, index creation and query inspection
Browser developer tools	Front-end debugging and response-time measurement for ESIMS-NF-001
